



A correction needs technical, operational and governance owners because the old claim may persist beyond its source record. AI-generated editorial image; not documentary evidence.

# The Source Was Corrected. Did the AI Stop Repeating the Old Answer?

**A correction-propagation contract for self-storage artificial intelligence across source records, retrieval indexes, caches, saved answers and pending actions.**

By Jared Mastroianni

Consider a fictional self-storage portfolio. A facility manager corrects a work order at 10:06 a.m.: the west vehicle gate is operating normally; the failed component was on the east pedestrian gate. The source record now shows the right asset.

At 10:11, an artificial intelligence assistant still tells a regional operator that the west vehicle gate is unavailable. At 10:14, a saved morning summary repeats the same statement. At 10:18, a pending workflow proposes sending customers to another entrance.

Nothing in this example requires a malicious model or a broken database. The source was corrected, but the old statement survived in derivative systems.

A correction is not complete when one row changes. It is complete when the organization knows which downstream uses received the old state, prevents new decisions from relying on it, updates or suppresses eligible derivatives, preserves the necessary history, and verifies what the AI now says.

That requires a **correction-propagation contract**: a bounded record connecting the correction to every material surface that can still retrieve, repeat, summarize or act on the old claim.

## A source edit and an AI correction are different events

Traditional applications often read a current field directly from a database. Many AI systems use a longer path. A source document may be copied into an ingestion store, divided into chunks, transformed into embeddings, added to a retrieval index, summarized, cached, placed into a conversation, exported to a report or used to form a pending action.

Changing the source does not prove that each derivative changed with it.

The architecture should therefore keep two events distinct:

- **Source correction**: an authorized record changes from one version to another.
- **Propagation closure**: every in-scope AI surface has been updated, invalidated, quarantined, superseded or explicitly accepted with a documented limitation.

The first event supplies the new fact. The second controls the old fact's remaining reach.

This distinction also protects the source system. An AI engineer should not rewrite an operating record simply to make a retrieval test pass. The source owner controls the record. The AI path consumes the authorized correction and proves how it handled the change.

## Begin with correction authority

Human feedback is important, but not every correction request is authoritative. A customer, employee, vendor or model may identify a conflict. The request still needs a named owner who can establish the governing record and permitted scope.

Before propagation begins, record:

1. **Corrected object:** the facility, asset, customer-neutral procedure, policy or observation being changed.
2. **Source locator and version:** the exact record, document or dataset version that governs the correction.
3. **Previous statement:** the claim that may remain in downstream systems.
4. **Replacement statement:** the authorized current claim, including its effective time.
5. **Correction type:** factual correction, permission change, expiration, retraction, scope clarification or superseding decision.
6. **Authority:** the person or role permitted to approve the change.
7. **Evidence:** the inspection, document, transaction or other basis for the correction.

“A manager said it was wrong” may justify a hold. It does not automatically establish the replacement fact. The system should be able to pause consequential use while the source owner verifies the issue.

## Map the derivative surfaces

The correction owner needs a map of every surface where the old statement may persist. The map should be specific to the use case, not a generic inventory of every technology in the company.

For a retrieval-augmented assistant, material surfaces may include:

- the authoritative source record;
- ingestion copies and parsed text;
- chunks and embeddings;
- the vector or keyword index;
- retrieval-result caches;
- final-answer caches;
- saved conversations and pinned answers;
- generated shift briefs, reports or emails still in draft;
- pending agent proposals or approval queues;
- dashboards or analytics built from generated labels;
- evaluation sets and test fixtures; and
- datasets approved for future training or fine-tuning.

Not every system contains every surface. The contract should say **not applicable** where a layer does not exist and **not evaluated** where its state is unknown. Neither condition is a pass.

The map should also identify the relationship type. Some derivatives contain the old text. Others contain a summary, classification, score or proposed action. Searching only for the original sentence can miss the most consequential copy.

## Treat correction types differently

Different corrections require different technical responses.

## Factual correction

The source statement was inaccurate. Replace the eligible indexed representation, invalidate affected caches, re-evaluate pending outputs and test the replacement fact.

## Permission change

The content may still be accurate, but a requester no longer has authority to retrieve it. Remove or restrict the content from the affected retrieval scope, invalidate permission-sensitive caches, and review saved derivatives under the approved access and retention policy.

## Expiration

The statement was valid for a defined period and is no longer current. Preserve the historical version, but prevent it from answering a current-state question without its time boundary.

## Retraction

The source owner withdraws the statement without supplying a replacement. The correct AI behavior may be “the prior statement was retracted; current status is unknown,” not a confident substitute.

## Scope clarification

The statement remains valid, but only for a narrower asset, facility, audience or use. Update metadata and retrieval filters so the claim cannot silently expand again.

One generic “refresh index” button cannot prove that all five conditions were handled correctly.

## Do not turn deletion into the only control

Some records must be retained for audit, safety, legal, contractual or operational reasons. A correction process should not erase history merely to make the current answer look clean.

Use explicit dispositions:

- **Update:** replace an eligible derivative with the corrected representation.
- **Invalidate:** prevent a cache or generated artifact from serving again.
- **Quarantine:** hold a derivative whose scope or effect is uncertain.
- **Supersede:** preserve the prior version while marking the authorized successor.
- **Restrict:** change who may retrieve or view the material.
- **Retain as history:** preserve the record but exclude it from current-state answers.
- **Escalate:** transfer unresolved retention, privacy, security or contractual questions to the proper owner.

The operating objective is not “delete every trace.” It is “prevent the old claim from controlling an unauthorized or current decision while preserving required evidence.”

## Pending actions need a stop condition

The highest-risk derivative may not be an answer. It may be an action waiting for approval.

When a material correction opens, the system should identify pending proposals that used the affected source version. Those proposals move to a defined state such as **held for revalidation**. They cannot become executable merely because a reviewer opened them before the correction arrived.

Completed actions are different. A source correction does not provide a universal undo command. A message already sent, access decision already applied or vendor dispatch already released requires its own authorized correction workflow. The propagation contract should link that workflow rather than claiming the action disappeared.

For consequential self-storage uses, the stop condition is straightforward: if the system cannot establish whether a pending proposal depends on the corrected claim, it should not release the proposal as current.

## Verify the whole path

Propagation is complete only after readback. Each target needs a test appropriate to its function.

**Source test:** the governing record returns the corrected version, authority and effective time.

**Ingestion test:** the parsed representation matches the eligible source content and version.

**Index test:** current retrieval returns the replacement statement or explicit unknown state; the superseded representation is excluded from current use.

**Cache test:** a query that previously hit the old answer either misses the invalidated entry or returns a newly generated corrected answer under the same permission scope.

**Conversation test:** saved context is labeled, restricted, superseded or re-evaluated according to policy. Starting a new chat is not proof that the old saved chat is safe.

**Action test:** pending proposals that used the old version are held, regenerated or cancelled before approval.

**Negative test:** the old claim does not return through alternate wording, another requester role or another facility with a similar name.

**Boundary test:** the correction does not overwrite a different asset, facility, time period or organization.

Store the query, requester scope, environment, source version, index version, cache state, result, reviewer and timestamp. A screenshot alone rarely proves which retrieval path or version produced an answer.

## A fictional correction run

The following example is fictional. It does not describe a customer, facility, deployment, product capability or measured result.

At 10:06 a.m., the facility operations owner corrects work order **WO-1842** from “west vehicle gate unavailable” to “east pedestrian gate closer requires adjustment.” The owner records the asset IDs, inspection note, effective time and reason for the correction.

The AI correction owner opens **COR-20261002-07**. Lineage shows that the prior work-order version entered an ingestion copy, two index chunks, an answer cache, a saved regional briefing and one pending customer-routing proposal.

The source and ingestion copy are updated. The two old chunks are retired from current retrieval and replaced with versioned chunks tied to the correct asset. The affected answer cache is invalidated. The regional briefing is preserved as a historical artifact but receives a visible superseded status and a link to the correction. The pending routing proposal moves to **held for revalidation** because it relied on the wrong gate.

Verification then runs under the regional operator's actual facility scope. Direct and paraphrased questions return the east pedestrian-gate condition and do not describe the west vehicle gate as unavailable. A same-name test at another fictional facility remains unchanged. The proposal is regenerated from the corrected source and reviewed before any release.

The contract closes at 10:32 a.m. It does not claim that every AI system in the company was corrected. It names the exact use case, surfaces, versions and readbacks that were in scope.

## Use a correction-propagation register

The companion register turns this process into an operating control. One row represents one correction target, not the correction as a whole. A correction with six affected surfaces has six target rows joined by one correction ID.

At minimum, each row records:

- correction ID and type;
- source locator, prior version and replacement version;
- affected facility, asset, audience and use;
- target surface and derivative identifier;

- relationship to the old claim;
- required disposition;
- owner and deadline;
- propagation status;
- verification query or check;
- result and evidence reference;
- exception owner and operating boundary; and
- closure authority and time.

The correction closes only when every required target is verified, explicitly out of scope, or held under an approved exception that states what the AI may do meanwhile.

This control is unnecessary for an inconsequential draft answer that is discarded and cannot be retrieved or acted on. It is appropriate when the old claim can still influence customer access, facility status, safety escalation, vendor work, financial treatment, employee evaluation or another consequential operating decision.

## Measure closure, not confidence

Useful measures include propagation completeness, correction latency by surface, number of pending actions held, number of derivative artifacts superseded and number of targets still not evaluated. None of those measures proves that the AI is accurate in general.

Model confidence is not a correction test. A fluent answer can confidently repeat a stale derivative. The relevant question is whether the response is grounded in the authorized current version for the requester's scope and use.

The National Institute of Standards and Technology (NIST) AI Risk Management Framework calls for post-deployment monitoring that includes feedback, appeal and override, incident response, recovery and change management.[1] Its Generative AI Profile adds suggested actions for ongoing monitoring, incident planning, system changes and provenance.[2] The Open Worldwide Application Security Project's retrieval-augmented generation guidance specifically identifies stale permission enforcement and persistent poisoning risks in cached responses, and recommends cache invalidation when source documents change.[3]

Those sources provide voluntary or community guidance across broader AI and security contexts. They do not validate a self-storage product, require this exact register or prove that any named business has deployed the architecture described here.

A corrected source is the beginning of the repair. A trustworthy AI workflow must also find the old claim's descendants, stop them from driving new decisions, preserve the right history and prove the current answer from the current source.

## Sources

1. National Institute of Standards and Technology, "AI Risk Management Framework Core," including Manage 4.1, accessed October 2, 2026: <https://airc.nist.gov/airmf-resources/airmf/5-sec-core/>
2. National Institute of Standards and Technology, *Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile*, NIST AI 600-1, July 2024, accessed October 2, 2026: <https://doi.org/10.6028/NIST.AI.600-1>
3. Open Worldwide Application Security Project, "Retrieval-Augmented Generation Security Cheat Sheet," accessed October 2, 2026: [https://cheatsheetseries.owasp.org/cheatsheets/RAG\\_Security\\_Cheat\\_Sheet.html](https://cheatsheetseries.owasp.org/cheatsheets/RAG_Security_Cheat_Sheet.html)

## Author

Jared Mastroianni is Chief Operating Officer of modSTORAGE and CEO and Founder of Facily.ai. His work focuses on facility operations, multi-location systems, data governance and responsible artificial intelligence.